SE(3)-WAM: Role-Aware 4D World–Action Modeling for Dynamic Manipulation

Anonymous CVPR submission

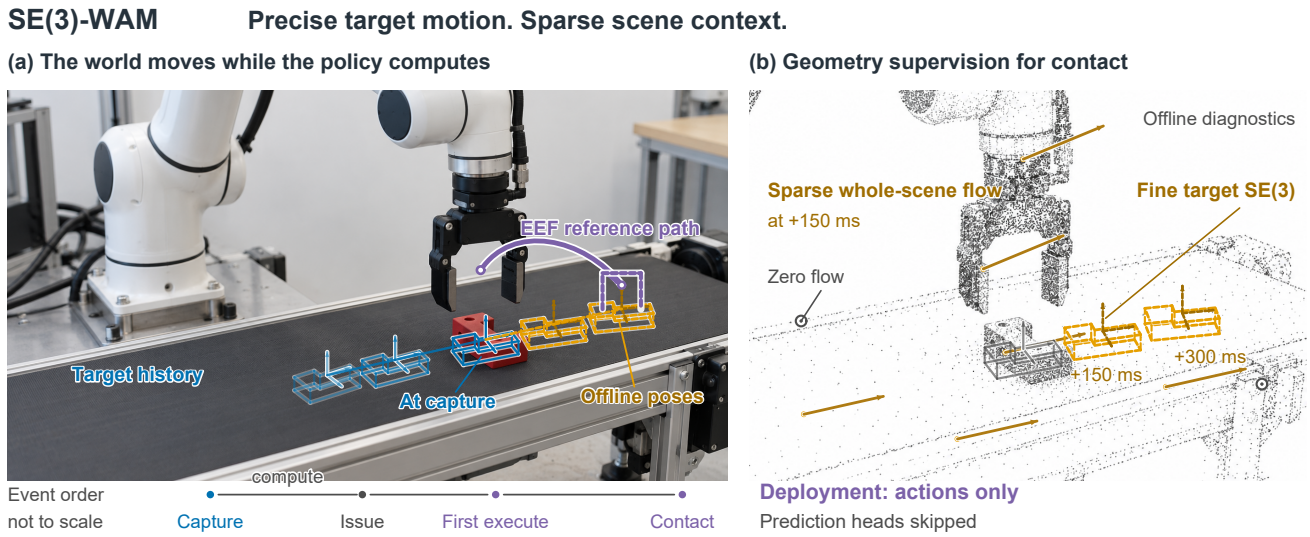


Figure 1. **Learning geometry for dynamic contact.** (a) Target history (blue), illustrative future poses (orange), and an end-effector reference path (purple) show motion between observation and contact. Capture, issue, first execution, and contact are distinct; the event strip is not to scale. (b) Fine target SE(3) and sparse whole-scene 3D flow provide training supervision. Horizons are measured from capture; flow is shown at $+150$ ms, with ringed zero-flow samples on static context. Background points depict scene context, not the supervision grid. Prediction heads are skipped at deployment; decoded geometry does not feed actions. *Design mockup: both scene images are generated capture-state context; poses, motion, and contact are illustrative, not trial data or model outputs.*

Abstract

Manipulating moving objects requires actions that fit the scene when they execute, not when the image was captured, yet every millisecond spent inferring the scene's motion lets the target move further. World–action models forecast future images or latents at uniform fidelity, spending time on appearance while leaving the target's metric pose implicit; target-centric predictors are fast but ignore the surroundings that constrain or drive the motion. We observe that the parts of a dynamic scene need different precision: the target's pose decides contact to within millimeters, the bodies around it matter only through the clearance they leave and the motion they impart to the target, and the remaining background does not matter. We present SE(3)-WAM, a world–action model built on this role-aware 4D structure. It encodes the target's motion as a metric SE(3) pose history and is trained to forecast the target's future SE(3) trajectory precisely and the whole scene coarsely as a sparse 3D flow. The forecasts serve only as training supervision and are skipped at deployment, so inference needs neither past frames nor a generated future and stays fast enough for real-time control. On Dynamic-14, a simulated benchmark in which the scene keeps moving during inference, SE(3)-WAM achieves [XX.X]% success, [XX.X] points above the strongest baseline, and succeeds in [XX]% of trials on six real-robot tasks with diverse target motion.

1. Introduction

Learning-based manipulation has advanced rapidly, from diffusion policies to vision–language–action (VLA) and world–action models (WAMs) [6, 7, 12, 20, 40, 46, 60], yet most of this progress assumes that objects stay still unless the robot moves them. In factories, kitchens, and human–robot handovers, objects keep moving while the robot perceives,

computes, and acts. This poses two coupled difficulties. A single image does not reveal how the target is moving, and by the time an action executes, the target is no longer where it was seen: a part on a conveyor at 0.1 m/s travels 15 mm during a 150 ms delay, more than a parallel-jaw grasp typically tolerates (Fig. 1).

Recent work equips policies with a memory of past motion and a forecast of the future [14, 26, 36, 53]. Both cost the resource that dynamic manipulation lacks most, time: richer histories and denser forecasts add computation, and the added latency enlarges the very error the forecast is meant to remove. Existing designs therefore trade information against speed. Video-based WAMs forecast whole future frames or latents at uniform fidelity [40, 46], also when conditioned on motion cues, which remain in image space and do not separate the target from other moving regions [26], and methods that restrict prediction to dynamic regions or salient tokens still predict in image space [36, 53]. In both, most capacity goes to appearance, and the quantity that decides contact, the target's metric pose when the action executes, stays implicit. Target-centric predictors and dynamic VLAs with object-level forecasts sit at the other end [14, 17, 27]: they are fast but blind to surroundings that can block or redirect the motion. None of them provides knowledge of the moving scene that is precise where contact depends on it, coarse elsewhere, and cheap to use at execution time.

Our key observation is that a dynamic policy needs to know very little precisely. Contact depends on the target's pose at the moment the action executes, to within millimeters. The bodies around it matter only through the clearance they leave and the motion they impart to the target, which a coarse estimate captures, and the rest of the background does not matter. Nor does this knowledge have to be computed at test time: a forecast that supervises training shapes what the policy represents and can be dropped at deployment [50]. SE(3)-WAM builds on both ideas (Fig. 2). It sees the current image together with short metric SE(3) histories of the target and the robot, which convey motion directly instead of a stack of past frames. A world stream is trained to forecast, in one metric 4D space, the target's future SE(3) trajectory precisely and the whole scene coarsely as a sparse 3D flow, and an action stream reads the world stream's summary and replans as often as its latency permits. SE(3)-WAM thus gains the precision of a metric forecast at a latency close to that of a reactive policy.

We evaluate SE(3)-WAM on Dynamic-14, a simulated benchmark of fourteen tasks in which observation, scene motion, and action execution share one physical clock, so the scene keeps moving while a policy computes. SE(3)-WAM achieves state-of-the-art success of [XX.X]%, [XX.X] points above the strongest baseline, at a p95 latency of [XX] ms, and succeeds in [XX]% of trials on six real-robot tasks, against [XX]% for the strongest baseline; it also leads on the 24 tasks of DOMINO [14]. Five final-scale controls isolate the contributions of target-motion history, future supervision, and its two geometric components, and compare role-aware with uniform-fidelity forecasts.

Our contributions are threefold:

- We propose role-aware 4D modeling for dynamic manipulation. Tracing how latency and estimation errors reach the contact shows that only the target needs millimeter precision, so we represent the target precisely by its metric SE(3) pose history and future trajectory, and the whole scene coarsely by a sparse 3D flow, in metric 3D rather than in image space.
- We present SE(3)-WAM, a world–action model trained with these role-aware 4D forecasts as supervision, at a latency close to that of a reactive policy.
- We build Dynamic-14, in which the scene keeps moving during inference, and show state-of-the-art results on it, on DOMINO, and on six real-robot tasks.

2. Related Work

Dynamic manipulation. Compensating for the time between sensing and acting is a classical problem, from control with dead time [33] to catching systems that track the target with Kalman-type filters and plan an interception [2–4, 29]; such systems are precise but engineered for each object and task. Learned policies have recently been brought to moving scenes [1]. PUMA reads historical optical flow and is supervised with future object features [14], DynamicWAM conditions a WAM on flow and kinematic descriptors and executes it with real-time chunking [26], and other methods forecast salient tokens, future images, or compact future codes and feed them to the policy at test time [27, 36, 51, 54]; DynaWM refines the chunks of a base VLA for moving objects [11], and DynamicManip synthesizes dynamic demonstrations from a static one [22]. Execution schemes instead reduce or hide latency, by overlapping inference with execution [31, 44], keeping consecutive chunks consistent [8, 24], or conditioning on the execution time [37]. They decide when actions are computed and applied, whereas SE(3)-WAM changes what the policy represents; we run every method under the same real-time executor and, as a control, with its original executor. DOMINO [14] and PhysMani-Bench [51] provide diverse dynamic-manipulation tasks, but their released default evaluation pipelines advance the simulation only after a policy has returned its prediction; Dynamic-14 advances the scene during inference and charges each method its measured latency.

Predictive supervision in world–action models. World–action models (WAMs) generate future video and actions in one network [21, 40, 46, 58], building on video prediction

as a policy objective [38, 43]. Because video is costly and dominated by appearance, recent work changes what is forecast, such as dynamic regions, depth, and semantic features [53], flow and geometry alongside RGB [49], object masks or slots [25, 47], or compact latents [57], and asks whether the forecast is needed at test time: Fast-WAM retains most of the benefit of video modeling without test-time imagination [50], and PUMA and others drop their predictors after training [14, 35, 55]. SE(3)-WAM adopts this training-only use and changes the target of the supervision: instead of pixels, features, or latents at one resolution, a metric 4D state whose precision follows each entity's role in contact.

Geometric representations of motion. Metric 3D inputs make policies more precise and sample-efficient [30, 52]. Motion has been represented by 2D point tracks [5, 42, 45], 3D flow [15, 18, 48], and object keypoints or SE(3) trajectories [17, 28, 34], and rigid-body decompositions have long been used to learn scene dynamics [9, 10]. These works describe the scene with one kind of representation, dense flow or tracks everywhere or a pose for each object; DOCIR separates the robot, the target, and obstacles, but as input encodings [13]. SE(3)-WAM assigns precision by role: a coarse sparse 3D flow describes the whole scene, and the target, on which contact depends to within millimeters, is additionally described by its SE(3) history and forecast.

3. SE(3)-WAM

SE(3)-WAM is a world–action model that represents a dynamic scene at role-dependent precision (Fig. 2). It encodes the current RGB-D frame, whose RGB a frozen MAE ViT-B/16 [16] turns into 196 patch tokens and whose depth gives each patch the metric 3D position of its center point in the robot base frame, together with short histories of the target's SE(3) pose and of the robot state. These 212 scene tokens and 16 learnable summary tokens that aggregate them form a world stream, which forecast heads supervise during training with the target's future SE(3) trajectory and a sparse 3D flow of the whole scene. An action stream shares attention with the world stream, reads its summary tokens, and generates action chunks by conditional flow matching [23].

3.1. Problem Formulation

SE(3)-WAM learns a policy from demonstrations. At query t, the policy receives an RGB image, a depth map, and the robot state captured at time τ_t, together with short histories of the robot state and of the pose $T(\tau) = (R(\tau), \mathbf{p}(\tau)) \in \mathrm{SE}(3)$ of a target object in the robot base frame. It outputs a chunk $(\mathbf{a}_{t,0}, \dots, \mathbf{a}_{t,H-1})$ of commands, one per control period Δ_a, indexed by time from capture: the j-th command is meant for

$$\tau^{\mathrm{a}}_{t,j} = \tau_t + j\Delta_a + d_{\mathrm{task}}. \tag{1}$$

Here d_{task} is a task's built-in command delay. The chunk becomes available only after a latency ℓ_t, so its commands with $j\Delta_a < \ell_t$ have expired, and the later ones execute at their times $\tau^{\mathrm{a}}_{t,j}$. A command must therefore fit the target's pose at $\tau^{\mathrm{a}}_{t,j}$ rather than at τ_t, a pose that the policy must estimate from observations at τ_t, and the lead $\Delta = \tau^{\mathrm{a}}_{t,j} - \tau_t$ of every executed command is at least ℓ_t, so it grows with every millisecond of inference. The bodies of a scene play different roles. The *target* is the body whose pose must be matched within a contact or containment tolerance ε, such as an object to grasp or a receptacle to place into. The *context* consists of the bodies that constrain or drive the target's motion: obstacles, such as an occluder or a wire, must be avoided with a clearance m, and drivers, such as a conveyor, a ramp, or a striker, move the target. The rest of the scene does not affect the task. Sec. 3.2 traces how the error of this estimate depends on the lead and on these roles, and derives the representation of SE(3)-WAM from it.

3.2. Role-Aware 4D Representation

What must be precise. Let a policy act, explicitly or implicitly, on an estimate of the target pose at τ^{a} with position error e and rotation error θ. A contact point at lever arm r is then off by at most $e + 2r\sin(\theta/2)$. Writing the target's motion over the lead as a constant velocity $\mathbf{v}$ plus a residual (acceleration, rolling, impacts), and the lead that the policy assumes as $\hat{\Delta}$, gives

$$e \le e_{\mathrm{loc}} + \hat{\Delta}\, e_v + \|\mathbf{v}\|\, e_\tau + e_{\mathrm{dyn}}, \tag{2}$$

with the errors e_{loc} of the current position, e_v of the velocity, $e_\tau = |\hat{\Delta} - \Delta|$ of the timing, and e_{dyn} of the residual motion (Appendix A.1). The tolerance ε is typically a few millimeters, and the rotation error is amplified by the lever arm, so the target's pose and motion must be known precisely. The context needs far less precision. An obstacle only has to be avoided with the clearance m, which is usually far larger than ε, and a driver matters through the motion it imparts to the target, which then appears in the target's own future; in both cases, a coarse estimate of the context suffices (Fig. 3c). We therefore represent the target by metric SE(3) states, both its observed history and its forecast trajectory, and the whole scene by a sparse 3D flow, all in the robot base frame and in physical time.

What latency allows. Latency enters Eq. (2) twice, through the lead that multiplies every velocity error and through the motion that the residual accumulates. By Eq. (2), information that costs inference time pays off only if it removes more error than the added latency creates. SE(3)-WAM keeps its inputs cheap, with one current image and low-dimensional state histories instead of past frames, and uses its forecasts only to supervise training, so that they add no inference time (Sec. 3.4).

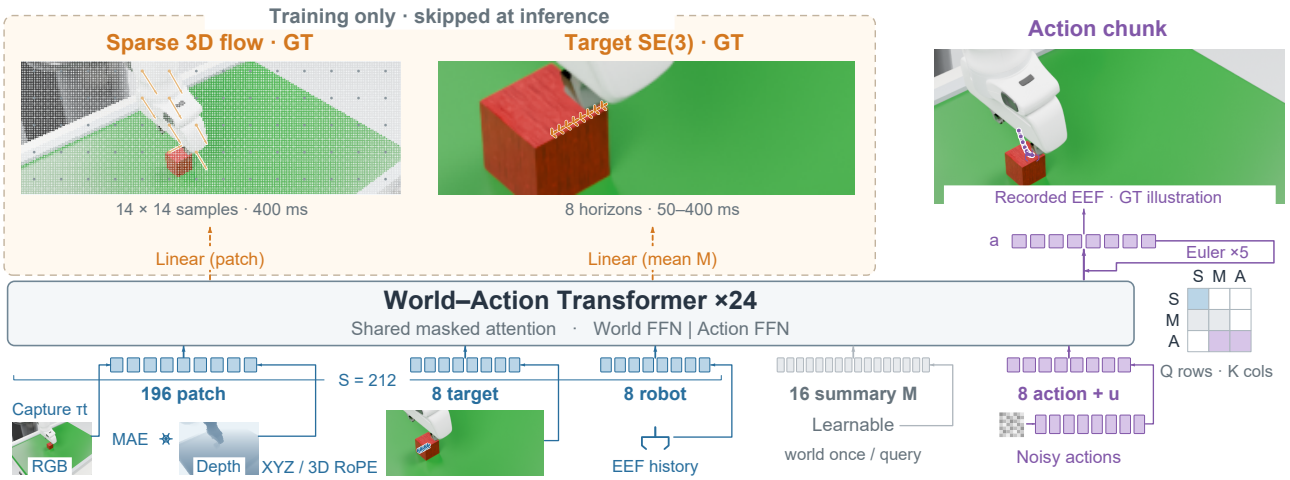


Figure 2. **SE(3)-WAM architecture.** Read from bottom to top. RGB-D and target/robot histories form $S = 212$ scene tokens. One backbone shares masked attention; actions read summary tokens M. Training-only linear heads supervise sparse whole-scene 3D flow/visibility from patches and target SE(3) from the mean summary. Geometry is GT from one recorded Dynamic-14 conveyor episode: a workspace crop of the 14×14 grid at 400 ms and eight orange pose axes at 50–400 ms. Blue axes show history; the purple path illustrates actions with recorded expert EEF states. RGB scenes are Blender renders; none of these illustrations is a model prediction. At inference, the entire dashed region is skipped and world keys/values are cached across 5 action Euler steps.

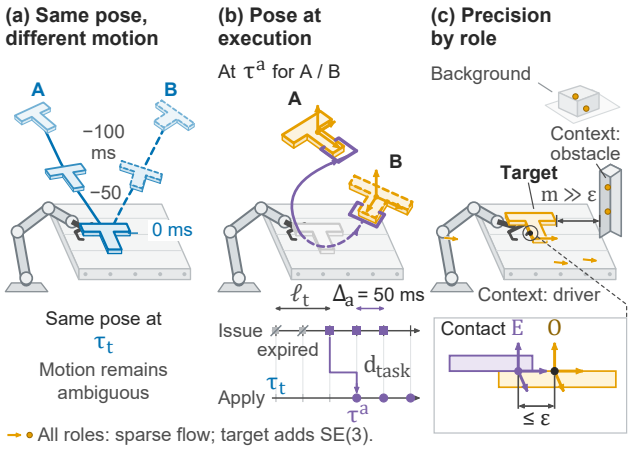


Figure 3. **Acting on a moving target** (schematic). (a) Different histories reach the same captured pose at τ_t. (b) At the same execution time τ^{a}, target translation and rotation differ, requiring different commands (purple). Issue slots are spaced by $\Delta_a = 50$ ms from capture. Gray slots expire during latency ℓ_t; retained commands apply after task delay d_{task}, first at $\tau^{\mathrm{a}} = \tau_t + \ell_t + d_{\mathrm{task}}$. (c) Contact aligns E and O within millimeter-scale ε; an obstacle leaves clearance $m \gg \varepsilon$, a conveyor drives the target, and distant background does not affect the task. Sparse flow covers all roles (arrows: displacement; dots: zero); the target additionally receives precise SE(3) supervision. Poses and delays are illustrative ($\ell_t = 2\Delta_a$, $d_{\mathrm{task}} = \Delta_a$); contact is enlarged.

Target motion: history and forecast. A single pose does not reveal how the target moves (Fig. 3a). The target history covers the last 8 control steps (350 ms at 20 Hz). It consists of the current target pose in the base frame and the seven interval motions between consecutive samples, each with its translation, relative rotation, time interval, and finite-difference velocity and acceleration in physical units, and forms 8 target-state tokens (Appendix A.2). These tokens state directly the velocity and acceleration on which e_v and part of e_{dyn} depend, at the cost of a few low-dimensional tokens. The robot history adds 8 end-effector state tokens. In simulation, the target pose comes from the simulator; on the real robot, a causal tracker estimates it from the RGB-D stream (Appendix D).

The forecast describes where the target will be when the commands execute (Fig. 3b). It covers fixed physical times $\tau_t + \delta_h$, with $\delta_h = h \cdot 50$ ms and $h = 1, \ldots, 8$, from 50 ms to 400 ms after capture, so that every command time $\tau^{\mathrm{a}}_{t,j}$ with $j \geq 1$ is a forecast horizon. Like all our motion labels, it is cumulative from the current state, and every loss averages over valid labels only. For each horizon, the target's motion is

$$\mathbf{d}_h = \mathbf{p}(\tau_t + \delta_h) - \mathbf{p}(\tau_t), \quad \boldsymbol{\phi}_h = \mathrm{Log}\big(R(\tau_t + \delta_h) R(\tau_t)^\top\big)^\vee. \tag{3}$$

A linear pose head on the mean of the final summary tokens predicts $(\hat{\mathbf{d}}_h, \hat{\boldsymbol{\phi}}_h)$ with the loss

$$\mathcal{L}_P = \frac{1}{|\mathcal{V}|} \sum_{h \in \mathcal{V}} \Big[\|\hat{\mathbf{d}}_h - \mathbf{d}_h\|^2 + \lambda_R \, d_{\mathrm{SO(3)}}\big(\hat{R}_h, R_h\big)^2 \Big], \tag{4}$$

where $\hat{R}_h = \mathrm{Exp}(\hat{\boldsymbol{\phi}}_h^\wedge) R(\tau_t)$ and $\mathcal{V}$ is the set of valid horizons. Because the pose head reads the summary tokens, the only world tokens that the actions attend to (Sec. 3.3), the

precise target supervision shapes exactly the representation that the actions use. The past and the future of the target thus share one metric frame and one time axis.

Scene motion: sparse 3D flow. The center points of the 196 patches on the 14×14 grid cover the target, the robot, and the surroundings. For each point i and horizon h, a linear head on its final patch token predicts the base-frame displacement $\hat{F}_{ih} \in \mathbb{R}^3$ and a visibility logit $\hat{o}_{ih}$, and

$$\mathcal{L}_F = \frac{1}{|\mathcal{V}_F|} \sum_{(i,h)\in\mathcal{V}_F} \Big[\tfrac{1}{3} \big\| (\hat{F}_{ih} - F_{ih})/s \big\|^2 + \mathrm{BCE}(\hat{o}_{ih}, o_{ih}) \Big], \tag{5}$$

with a fixed scale s fitted on the training labels, the visibility o_{ih}, and the set $\mathcal{V}_F$ of valid labels. The grid samples the workspace sparsely, about [X] mm between points. The flow thus gives the context and the robot a motion estimate at this spacing and stays near zero on the static background, while the target additionally receives the dedicated SE(3) supervision of Eq. (4). The flow supervises the patch tokens from which the summaries draw, one level below the precise target supervision; this is how SE(3)-WAM allocates precision by role.

Auxiliary supervision. Two further targets regularize the representation; we selected them on development scenes (Appendix E). Future appearance heads on the patch tokens predict, at every horizon, the metric depth, the features of a frozen DINOv3 teacher [32], and the RGB of moving surfaces. Separately, a small head on the summary tokens reconstructs the RGB of the current target region, which ties the summaries to the target without a future label. Their losses form $\mathcal{L}_{\mathrm{aux}}$ (Appendix A.2).

3.3. Action Generation and Joint Learning

Action stream. The action stream holds $H = 8$ tokens, one per command of the chunk; each embeds a noisy command, an end-effector translation and rotation increment plus a gripper command (Appendix A.2). The attention mask $\mathcal{G}$ lets scene tokens attend to the scene, summary tokens to the scene and the summaries, and action tokens to the summaries and the actions, so the actions read the world through the summary tokens and the world stream does not depend on the actions. In every layer, both streams share the attention projections, through which they exchange information, and keep separate feed-forward networks, which give each its own capacity; we selected this backbone over fully separate parameters on development scenes (Appendix E.4). World tokens use layer normalization, and action tokens use adaptive layer normalization conditioned on the flow time.

Joint learning. For a chunk $\mathbf{a}_1$ normalized with training statistics (Appendix A.2), noise $\mathbf{a}_0 \sim \mathcal{N}(0, I)$, and flow time $u \sim \mathcal{U}[0, 1]$, the action stream v_ψ, with parameters ψ, regresses the flow-matching velocity

$$\mathcal{L}_A = \mathbb{E} \left\| v_\psi(\mathbf{a}_u, u \mid \mathcal{S}_t, \mathcal{G}) - (\mathbf{a}_1 - \mathbf{a}_0) \right\|^2, \tag{6}$$

with $\mathbf{a}_u = (1-u)\mathbf{a}_0 + u\mathbf{a}_1$, the scene tokens $\mathcal{S}_t$ of query t, and the error averaged over valid commands. All parts are trained jointly with

$$\mathcal{L} = \mathcal{L}_A + \lambda_P \mathcal{L}_P + \lambda_F \mathcal{L}_F + \lambda_{\mathrm{aux}} \mathcal{L}_{\mathrm{aux}}, \tag{7}$$

with the weights given in Sec. 4.1.

3.4. Efficient Inference and Execution

SE(3)-WAM trains its forecast heads jointly and skips them at test time, as recent WAMs do [21, 50]. Because world tokens attend neither to the actions nor to the flow time, the world stream runs once per query, and the action stream integrates 5 Euler steps on its cached keys and values. The executor counts latency in whole control periods, as RTC does [8]. A query of total latency L_t hands its chunk over at the first control-period boundary at or after it ends, $\ell_t = k_t \Delta_a$ after capture with $k_t = \lceil L_t/\Delta_a \rceil$, and execution continues from command k_t, the one meant for that boundary. There the policy observes again and starts the next query; until then, the executor continues the previous chunk. The policy therefore replans as often as its latency allows, before every command when a query fits in one control period. Because each command is issued at the time its label refers to, the action stream needs no latency input, and the timing error e_τ of Eq. (2) is limited to actuation delays. Every executed command, at most $(H-1)\Delta_a + d_{\mathrm{task}}$ after capture, lies within the horizons of the forecast supervision (Appendix B.3). On the robot, L_t also includes tracking the target in the newest frame.

4. Experiments

We evaluate how SE(3)-WAM performs when the scene keeps moving during inference (Sec. 4.2), what each of its components contributes (Sec. 4.3), and how it transfers to a real robot (Sec. 4.4).

4.1. Setup

Dynamic-14. Dynamic-14 is a simulated benchmark in which observation, scene motion, and action execution share one physical clock, so the scene keeps moving while a policy computes and every method pays for its own latency. It contains fourteen tasks in MuJoCo/robosuite [39, 59] in five families: grasping a moving target (harmonic, conveyor, translating, rotating, and occluded motion), placing into a moving receptacle, passing a wand along a rotating wire,

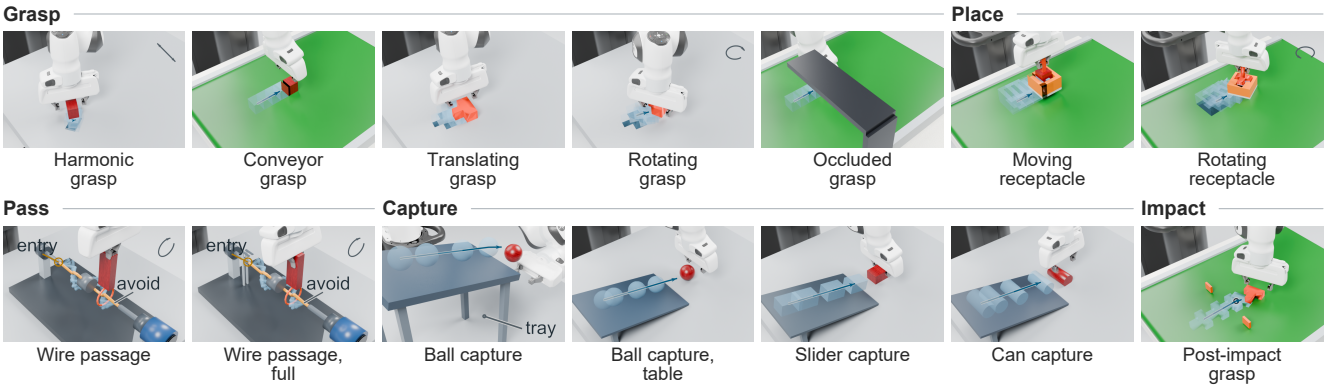


Figure 4. **Dynamic-14.** Fourteen tasks in five families, rendered from recorded simulator states with a common agent view. Four translucent blue poses fade from recent to earlier history; navy traces follow measured past motion. Gray corner arrows indicate rotation or oscillation qualitatively. A small ring marks the target center during recorded impact. Both Pass panels show passage: the wire is a target at entry and an obstacle along the path; Full additionally requires acquisition, exit and release. Occluded motion stays hidden. These native-expert examples illustrate task dynamics, not SE(3)-WAM outcomes; no future prediction is shown.

capturing rolling or sliding objects, and grasping after an impact (Fig. 4; Appendix B). Experts with privileged simulator state provide 800 demonstrations per task. We select configurations on development scenes and report all Dynamic-14 results on 1,400 held-out scenes (100 per task, new seeds).

Evaluation. During each query, physics advances by the method's own inference latency, drawn from its latency distribution measured at batch size 1 on one RTX 5090, the GPU of our robot, independently of the hardware that runs the rollouts (Appendix B.3). All methods share one executor that indexes each chunk by time from capture (Sec. 3.4): a query starts at a control-period boundary, and its chunk takes over at the first boundary at or after the query ends (Appendix B.3). Besides this native-latency setting, we add 150 ms of latency to every method and pause physics during inference; Appendix C.2 adds equal-latency and original-executor controls. We report the success rate (SR), averaged over the fourteen tasks with equal weight and over training seeds, and the p95 latency, the measured time from an observation to its first command (time-to-action). Paired 95% intervals come from a bootstrap over held-out scenes.

Baselines. We compare with reactive policies, dynamic VLAs, image- or latent-space WAMs, and a dynamic WAM (Table 1). Each baseline is trained on the same demonstrations with its official code, inputs, and recipe, including a rendered wrist camera and frame or flow histories where the recipe uses them (Appendix B.5). SE(3)-WAM uses one agent view with its depth and the target's SE(3) history, without past frames. To give baselines the same target-motion information, $\pi_{0.5}$ and DynamicWAM also run with our SE(3) history tokens (+SE(3)).

Table 1. **Main results.** Task-balanced SR on 1,400 held-out Dynamic-14 scenes: native latency, +150 ms, and paused physics. Three training seeds for SE(3)-WAM, one per baseline. p95: time-to-action on RTX 5090. Wrist and history denote extra inputs (Sec. 4.1). Blue/bold and light blue mark the best and second reported values per column; rankings remain provisional while results are pending.

Method	Wrist	History RGB	History Flow	History SE(3)	SR (%) ↑ Native	SR (%) ↑ +150 ms	SR (%) ↑ Paused	p95 (ms) ↓
ACT [56]					**42.0**	**24.1**	**40.9**	**10.7**
DP [12]		✓			0.0	0.0	33.1	1701.9
$\pi_{0.5}$ [6]					—	—	—	—
$\pi_{0.5}$+SE(3)	—			✓	—	—	—	—
DynamicVLA [44]		✓			—	—	—	—
PUMA [14]			✓		—	—	—	—
DreamVLA [53]		✓			—	—	—	—
OpenWAM-α [40]					—	—	—	—
DynamicWAM [26]			✓		—	—	—	—
DynamicWAM+SE(3)	—		✓	✓	—	—	—	—
SE(3)-WAM				✓	—	—	—	—

Implementation. The transformer has 24 layers of width 1024 with 16 heads and 523.8M trainable parameters; the MAE ViT-B/16 encoder and the DINOv3 teacher stay frozen. On Dynamic-14, it trains from random initialization for 20 epochs with AdamW, $\lambda_P = \lambda_F = 1$, and $\lambda_R = \lambda_{\text{aux}} = 0.1$ (Appendix A.3). On the real robot, it is fine-tuned per task from a 48-layer base pretrained on DROID [19] (Appendix A.4).

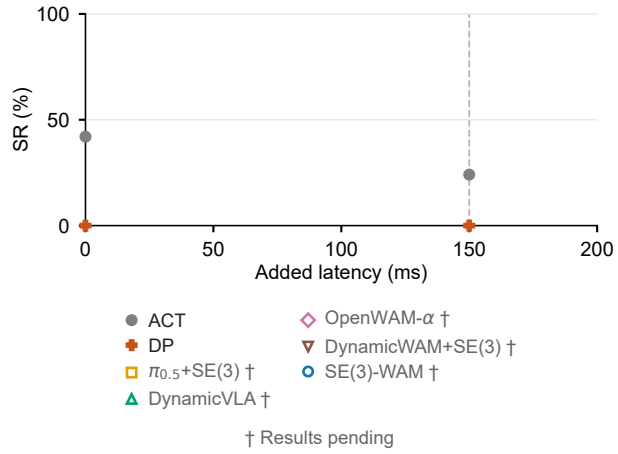


Figure 5. **Success under added latency.** Comparison on 1,400 held-out Dynamic-14 scenes. The legend retains all planned series, plus DP; † marks methods awaiting results. ACT and DP have measurements at 0/150 ms. Only adjacent measured points are connected. Dashed line: +150 ms; full sweep table: Table S7.

4.2. Simulation Results

Comparison. With its native latency, SE(3)-WAM reaches [XX.X]% SR on held-out Dynamic-14 scenes (Table 1), [X.X] points above the strongest baseline, [name], and [X.X] points above the strongest baseline that receives the same SE(3) history, [name]. Its gains are largest on [task types] (Appendix C.1). Its p95 latency of [XX] ms is [X]× lower than that of the image-space WAMs and close to that of [the reference reactive policy, to be fixed].

Latency and protocols. With 150 ms of added latency, SE(3)-WAM keeps [XX.X]% SR, whereas the reactive policies drop to [XX.X]% and the image-space WAMs to [XX.X]% on average, as Eq. (2) predicts: added latency lengthens the lead that multiplies every velocity error, so methods that do not anticipate the target's motion lose most. Fig. 5 shows the available measurements in the 0–200 ms sweep; pending points are listed in Appendix C.2. With physics paused during inference, which ignores the cost of computation, SE(3)-WAM still leads [name] by [X.X] points, against [X.X] points with native latency. When every method is charged the p95 latency of SE(3)-WAM, its margin is [X.X] points, and running [the baselines that rely on their own asynchronous execution] with their original executors changes their SR by at most [X.X] points (Appendix C.2).

DOMINO. We also train SE(3)-WAM, $\pi_{0.5}$, DynamicWAM, and PUMA on 24 single-arm Franka tasks of DOMINO [14] with its demonstrations and evaluate them with its native evaluator. SE(3)-WAM reaches [XX.X]% SR against [XX.X]% for [the strongest baseline] (Table 2; Appendix C.3). As on Dynamic-14, it reads the simulator pose of the target.

Table 2. **DOMINO results** on 24 single-arm clean-L1 tasks, each method trained on DOMINO's demonstrations (one training seed per method, 50 episodes per task and method). SR (%) is grouped by interaction type and averaged over tasks; MS is the manipulation score. Best per column will be bold on blue once measured. Protocol: Appendix C.3.

Method	Intercept	Track	All ↑	MS ↑
$\pi_{0.5}$	—	—	—	—
DynamicWAM	—	—	—	—
PUMA	—	—	—	—
SE(3)-WAM	—	—	—	—

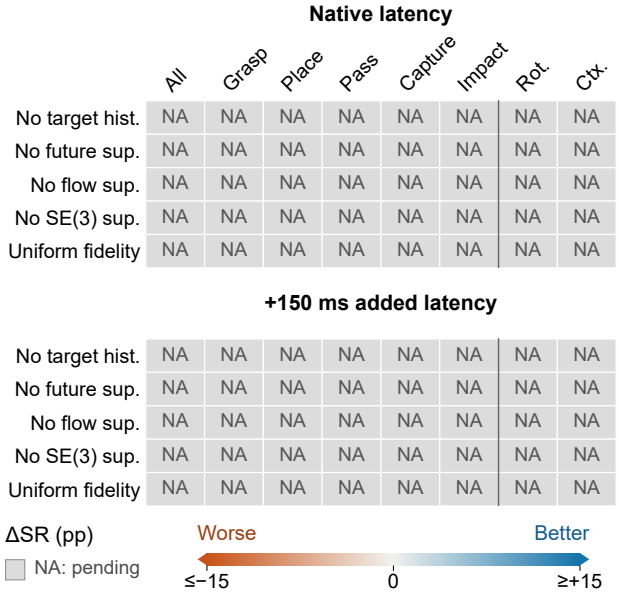


Figure 6. **Ablation effects by task (design preview; data pending).** ΔSR = SR(control) − SR(full), in percentage points; colors saturate at ±15. Rows: the five controls of Sec. 4.3; the full model reaches [XX.X]% SR with native latency and [XX.X]% with +150 ms. Rot./Ctx.: predeclared, overlapping rotation-sensitive/context-active groups (Table S3). Planned estimates average three seeds per scene and weight tasks equally, retaining `t3_phase`. Bold will mark unadjusted paired 95% scene-bootstrap intervals excluding zero, conditional on trained weights (Appendix B.4). Grey NA: no eligible paired results or estimates.

4.3. Ablations and Analysis

Five controls each change one component of the final model and keep the data, recipe, and training budget, with three training seeds each (Fig. 6). They test the argument of Sec. 3.2: the target history supplies the velocity on which e_v in Eq. (2) depends, future supervision shapes the representation without adding inference time, and the two geometric

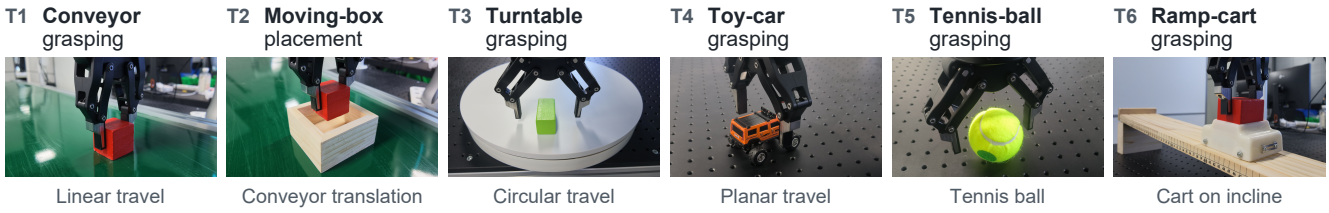


Figure 7. **Real-robot task setups.** Photographs of conveyor grasping (T1), placement into a moving box (T2), turntable grasping (T3), moving toy-car grasping (T4), tennis-ball grasping (T5), and grasping a block carried by a ramp cart (T6).

Table 3. **Real-robot results.** Success rate (%) on the six tasks of Sec. 4.4, 20 trials per task and method; All pools the trials. p95 lat.: p95 latency, the end-to-end time-to-action (ms) on one RTX 5090, including tracking for SE(3)-WAM. DP: Diffusion Policy. Once measured, best per column will be bold on blue, including ties. Successes, intervals, and failures in Appendix D.

Method	T1	T2	T3	T4	T5	T6	All ↑	p95 (ms) ↓
$\pi_{0.5}$	—	—	—	—	—	—	—	—
DynamicWAM	—	—	—	—	—	—	—	—
DP	—	—	—	—	—	—	—	—
SE(3)-WAM	—	—	—	—	—	—	—	—

components and a uniform-fidelity forecast test the allocation of precision by role. We evaluate them with native latency and with 150 ms of added latency (per-seed results and a sensitivity analysis in Appendix C.4); since the forecast heads are skipped at deployment, all controls share the inference cost of SE(3)-WAM.

Target history. The no-history control removes the past target poses with their velocity and acceleration features and keeps the current image, the current target pose, and the robot history; unlike the other controls, it changes the input. Without the history, SR drops by [X.X] points with native latency and by [X.X] points with 150 ms of added latency, as Eq. (2) predicts: a longer lead amplifies the velocity error that the history removes. Because the history relies on pose estimates, we also perturb them; under synthetic correlated pose noise, SE(3)-WAM keeps [XX.X]% SR (Appendix C.5).

Future supervision. The no-future control removes all forecast losses, the SE(3), scene-flow, and future appearance targets, and keeps the current-target RGB reconstruction. Its inference is identical to that of SE(3)-WAM, yet SR drops by [X.X] points with native latency and by [X.X] points with 150 ms of added latency, most on [tasks with non-uniform target motion]. Offline, the learned SE(3) forecast is also more accurate than constant-velocity and constant-acceleration extrapolation of the same history (Table S9).

Allocation of precision. Three controls keep all auxiliary targets and change the geometric forecast. Removing the scene-flow supervision lowers SR by [X.X] points; because the flow also covers the target and the robot, and the appearance targets remain, this measures the contribution of coarse scene-motion supervision rather than of all context information. Removing the dedicated SE(3) supervision leaves the target with only the coarse flow that every part of the scene receives and lowers SR by [X.X] points. If precision follows the roles, the SE(3) supervision should matter most on tasks with a tight pose tolerance or a rotation-sensitive contact, and the flow supervision on tasks whose context constrains or drives the target. [Whether Fig. 6 shows this pattern; if it does not, state that the controls establish the contribution of each component but not the allocation by role.] The uniform-fidelity control replaces the SE(3) and scene-flow targets by [future latents of a frozen video VAE] of the whole frame at the same horizons, so that every part of the scene is forecast at one fidelity; at the same inference cost, it trails SE(3)-WAM by [X.X] points. [One-sentence summary of which components matter, and where.]

4.4. Real-Robot Experiments

Setup. We evaluate six tasks on a Franka arm ([model]) with [gripper model] and a fixed RealSense D435 RGB-D camera (Fig. 7; Appendix D): conveyor grasping (T1), placement into a moving receptacle (T2), turntable grasping (T3), moving toy-car grasping (T4), tennis-ball grasping (T5), and block grasping from a ramp cart (T6). Baselines are $\pi_{0.5}$ (VLA), DynamicWAM (dynamic WAM), and Diffusion Policy [12] (reactive). All methods train or fine-tune on the same 100 teleoperated demonstrations per task and run 20 trials per task with shared trial keys (Appendix D.3). Baselines receive RGB [and a wrist camera if their recipe uses one]; SE(3)-WAM also receives FoundationPose [41] target estimates, updated at every control-period boundary before its query (Appendix D.2). End-to-end latencies include tracking (Table 3).

Results. SE(3)-WAM succeeds in [XX.X]% of all trials, against [XX.X]% for [the strongest baseline], a paired difference of [X.X] points (95% interval [[X.X, X.X]]; Table 3).

Its p95 latency, including tracking, is [XX] ms, [within one control period], whereas [name] needs [XX] ms. The margin is largest on [tasks], and the dominant failure of SE(3)-WAM is [failure mode] (Appendix D.4).

5. Conclusion

SE(3)-WAM spends precision where contact depends on it: a metric SE(3) history and forecast of the target and a sparse 3D flow of the whole scene, with the forecasts used only as training supervision. It reaches [XX.X]% success on Dynamic-14, [XX.X]% on DOMINO, and [XX]% on six real-robot tasks at a p95 latency of [XX] ms, improving on reactive, dynamic, and world–action policies. [The ablations show that the target history matters most under added latency and that future supervision improves control without changing the inference computation.] [State what the dedicated SE(3), scene-flow, and uniform-fidelity controls support: a better forecast design, or, if the task groups also agree, precision allocated by role.] Accounting for inference latency [state how success rates and relative performance differ from paused evaluation].

Limitations. SE(3)-WAM requires a specified rigid target and reliable estimates of its pose, and its forecast does not model how the robot's own contact changes the target's motion. Its target-trajectory head predicts a single future per horizon and does not represent uncertainty or multimodal motion, which an uncertainty-aware forecast could address.

LLM Usage Statement

We used OpenAI Codex to assist with literature review, code implementation and debugging, experiment execution, and language editing of the manuscript. Codex also assisted in developing plotting code and refining figure and manuscript layouts. The authors reviewed AI-assisted content, checked references against original sources, and validated the implementation through testing and experiments. The authors take full responsibility for the final manuscript, experimental results, and conclusions.

References

[1] Iretiayo Akinola, Jingxi Xu, Shuran Song, and Peter K. Allen. Dynamic grasping with reachability and motion awareness. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 9422–9429, 2021. 2

[2] Peter K. Allen, Aleksandar Timcenko, Billibon Yoshimi, and Paul Michelman. Automated tracking and grasping of a moving object with a robotic hand-eye system. *IEEE Transactions on Robotics and Automation*, 9(2):152–165, 1993. 2

[3] Yaakov Bar-Shalom, X. Rong Li, and Thiagalingam Kirubarajan. *Estimation with Applications to Tracking and Navigation: Theory, Algorithms and Software*. Wiley, 2001.

[4] Berthold Bäuml, Thomas Wimböck, and Gerd Hirzinger. Kinematically optimal catching a flying ball with a hand-arm-system. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2010. 2

[5] Homanga Bharadhwaj, Roozbeh Mottaghi, Abhinav Gupta, and Shubham Tulsiani. Track2Act: Predicting point tracks from internet videos enables generalizable robot manipulation. In *European Conference on Computer Vision*, 2024. 3

[6] Kevin Black, Noah Brown, James Darpinian, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. In *Proceedings of The 9th Conference on Robot Learning*, pages 17–40, 2025. 1, 6

[7] Kevin Black, Noah Brown, Danny Driess, et al. π_0: A vision-language-action flow model for general robot control. In *Proceedings of Robotics: Science and Systems*, 2025. 1

[8] Kevin Black, Manuel Galliker, and Sergey Levine. Real-time execution of action chunking flow policies. In *Advances in Neural Information Processing Systems*, pages 33383–33407, 2025. 2, 5

[9] Arunkumar Byravan and Dieter Fox. SE3-Nets: Learning rigid body motion using deep neural networks. In *IEEE International Conference on Robotics and Automation*, pages 173–180, 2017. 3

[10] Arunkumar Byravan, Felix Leeb, Franziska Meier, and Dieter Fox. SE3-Pose-Nets: Structured deep dynamics models for visuomotor control. In *IEEE International Conference on Robotics and Automation*, 2018. 3

[11] Chongkei Chang and Zhidong Deng. DynaWM: A base-VLA-guided world foundation model for moving-object manipulation. *arXiv preprint arXiv:2607.02604*, 2026. 2

[12] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin C. M. Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Proceedings of Robotics: Science and Systems*, 2023. 1, 6, 8

[13] David Emukpere, Romain Deffayet, Bingbing Wu, Romain Brégier, Michael Niemaz, Jean-Luc Meunier, Denys Proux, Jean-Michel Renders, and Seungsu Kim. Disentangled object-centric image representation for robotic manipulation. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 15873–15879, 2025. 3

[14] Heng Fang, Shangru Li, Shuhan Wang, Xuanyang Xi, Dingkang Liang, and Xiang Bai. Towards generalizable robotic manipulation in dynamic environments. In *European Conference on Computer Vision*, pages 314–333, 2026. 2, 3, 6, 7

[15] Jun Guo, Xiaojian Ma, Yikai Wang, Min Yang, Huaping Liu, and Qing Li. FlowDreamer: A RGB-D world model with flow-based motion representations for robot manipulation. *IEEE Robotics and Automation Letters*, 11(3):2466–2473, 2026. 3

[16] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16000–16009, 2022. 3

[17] Cheng-Chun Hsu, Bowen Wen, Jie Xu, Yashraj Narang, Xiaolong Wang, Yuke Zhu, Joydeep Biswas, and Stan Birchfield. SPOT: SE(3) pose trajectory diffusion for object-centric manipulation. In *IEEE International Conference on Robotics and Automation*, 2025. 2, 3

[18] Wenlong Huang, Yu-Wei Chao, Arsalan Mousavian, Ming-Yu Liu, Dieter Fox, Kaichun Mo, and Li Fei-Fei. PointWorld: Scaling 3D world models for in-the-wild robotic manipulation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 20765–20779, 2026. 3

[19] Alexander Khazatsky, Karl Pertsch, Suraj Nair, et al. DROID: A large-scale in-the-wild robot manipulation dataset. In *Proceedings of Robotics: Science and Systems*, 2024. 6

[20] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, et al. OpenVLA: An open-source vision-language-action model. In *Conference on Robot Learning*, pages 2679–2713, 2025. 1

[21] Shuang Li, Yihuai Gao, Dorsa Sadigh, and Shuran Song. Unified video action model. In *Proceedings of Robotics: Science and Systems*, 2025. 2, 5

[22] Haoran Liao, Pengyue Wang, Shuoyu Chen, Kehan Cheng, Xuhang Chen, Yuhao Lin, Mu Lin, Zhizhao Liang, Xiaoyi Fan, Chengyi Xing, Dan Niu, Yi-Lin Wei, and Wei-Shi Zheng. DynamicManip: Enabling dynamic manipulation from a single static demonstration. *arXiv preprint arXiv:2608.01452*, 2026. 2

[23] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow matching for generative modeling. In *International Conference on Learning Representations*, 2023. 3

[24] Yuejiang Liu, Jubayer Hamid, Annie Xie, Yoonho Lee, Max Du, and Chelsea Finn. Bidirectional decoding: Improving action chunking via guided test-time sampling. In *International Conference on Learning Representations*, pages 4594–4627, 2025. 2

[25] Yushan Liu, Peibo Sun, Shoujie Li, Yifan Xie, Lingfeng Zhang, Xintao Chao, Shiyuan Dong, Fang Chen, Xiao-Ping Zhang, and Wenbo Ding. OA-WAM: Object-addressable world action model for robust robot manipulation. *arXiv preprint arXiv:2605.06481*, 2026. 3

[26] Yunfan Lou, Hewen Gao, Xiyu Zhu, Zhuoran Qiao, Xuan Han, Yifan Yang, Yifan Ye, Boxian Yao, and Zhibo Pang. DynamicWAM: Dual-path motion conditioning for world-action models in dynamic manipulation. *arXiv preprint arXiv:2608.00793*, 2026. 2, 6

[27] Siyuan Ma, Yutian Zhang, Boshi Zhang, Qinglian Wu, Jiaqi Zhai, Dong Wei, and Xiaojin Huang. ForeTime-VLA: Causal future-token distillation from a world action model for conveyor-belt manipulation. *arXiv preprint arXiv:2608.20735*, 2026. 2

[28] Lucas Manuelli, Wei Gao, Peter Florence, and Russ Tedrake. kPAM: Keypoint affordances for category-level robotic manipulation. In *Robotics Research: The 19th International Symposium ISRR*, pages 132–157. Springer, 2022. 3

[29] Naresh Marturi, Marek Kopicki, Alireza Rastegarpanah, et al. Dynamic grasp and trajectory planning for moving objects. *Autonomous Robots*, 43(5):1241–1256, 2019. 2

[30] Mohit Shridhar, Lucas Manuelli, and Dieter Fox. Perceiver-actor: A multi-task transformer for robotic manipulation. In *Conference on Robot Learning*, pages 785–799, 2023. 3

[31] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, et al. SmolVLA: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025. 2

[32] Oriane Siméoni, Huy V. Vo, Maximilian Seitzer, Federico Baldassarre, Maxime Oquab, Cijo Jose, Vasil Khalidov, Marc Szafraniec, Seungeun Yi, Michaël Ramamonjisoa, et al. DINOv3. *arXiv preprint arXiv:2508.10104*, 2025. 5

[33] Otto J. M. Smith. Closer control of loops with dead time. *Chemical Engineering Progress*, 53(5):217–219, 1957. 2

[34] Rustin Soraki, Homanga Bharadhwaj, Ali Farhadi, and Roozbeh Mottaghi. ObjectForesight: Predicting future 3D object trajectories from human videos. In *European Conference on Computer Vision*, pages 437–453, 2026. 3

[35] Yue Su, Sijin Chen, Haixin Shi, Mingyu Liu, Zhengshen Zhang, Ningyuan Huang, Weiheng Zhong, Zhengbang Zhu, Yuxiao Liu, and Xihui Liu. World guidance: World modeling in condition space for action generation. In *Proceedings of the 43rd International Conference on Machine Learning*, pages 115917–115933, 2026. 3

[36] Shahram Najam Syed, Arthur Jakobsson, Haoran Hao, and Jeffrey Ichnowski. Intercepting the future: Latent-space predictive world model for dynamic VLA manipulation. In *Conference on Robot Learning*, 2026. Accepted for publication. 2

[37] Jiaming Tang, Yufei Sun, Yilong Zhao, Shang Yang, Yujun Lin, Zhuoyang Zhang, James Hou, Yao Lu, Zhijian Liu, and Song Han. VLASH: Real-time VLAs via future-state-aware asynchronous inference. *arXiv preprint arXiv:2512.01031*, 2025. 2

[38] Yang Tian, Sizhe Yang, Jia Zeng, Ping Wang, Dahua Lin, Hao Dong, and Jiangmiao Pang. Predictive inverse dynamics models are scalable learners for robotic manipulation. In *International Conference on Learning Representations*, pages 92033–92052, 2025. 3

[39] Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033, 2012. 5

[40] Yuran Wang, Siqiao Huang, Mingleyang Li, Chenhao Zhang, Jiaqi Liang, Weiyang Jin, Yue Chen, Xuemin Chi, Donghao Zhou, Qize Yu, Yu-Kai Wang, Yuhan Rui, Shenzhe Yao, Zhen Yuan, Zhenhao Shen, Kefei Zhu, Zijie Zhu, Ning Gao, Xiaowei Chi, Guanqi He, Shanghang Zhang, Hao Dong, Lin Shao, and Hang Zhao. OpenWAM: An open, modular exploration towards systematic world-action model pretraining. *arXiv preprint arXiv:2609.07398*, 2026. 1, 2, 6

[41] Bowen Wen, Wei Yang, Jan Kautz, and Stan Birchfield. FoundationPose: Unified 6D pose estimation and tracking of novel objects. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 17868–17879, 2024. 8

[42] Chuan Wen, Xingyu Lin, John Ian Reyes So, Kai Chen, Qi Dou, Yang Gao, and Pieter Abbeel. Any-point trajectory modeling for policy learning. In *Proceedings of Robotics: Science and Systems*, 2024. 3

[43] Hongtao Wu, Ya Jing, Chilam Cheang, Guangzeng Chen, Jiafeng Xu, Xinghang Li, Minghuan Liu, Hang Li, and Tao Kong. Unleashing large-scale video generative pre-training for visual robot manipulation. In *International Conference on Learning Representations*, pages 10641–10662, 2024. 3

[44] Haozhe Xie, Beichen Wen, Jiarui Zheng, Zhaoxi Chen, Fangzhou Hong, Haiwen Diao, and Ziwei Liu. DynamicVLA: A vision-language-action model for dynamic object manipulation. In *Advances in Neural Information Processing Systems*, 2026. Accepted for publication. 2, 6

[45] Mengda Xu, Zhenjia Xu, Yinghao Xu, Cheng Chi, Gordon Wetzstein, Manuela Veloso, and Shuran Song. Flow as the cross-domain manipulation interface. In *Conference on Robot Learning*, pages 2475–2499, 2025. 3

[46] Seonghyeon Ye, Yunhao Ge, Kaiyuan Zheng, Shenyuan Gao, Sihyun Yu, George Kurian, Suneel Indupuru, et al. World action models are zero-shot policies. *arXiv preprint arXiv:2602.15922*, 2026. 1, 2

[47] Hanyang Yu, Haitao Lin, Jingbo Zhang, Wenyao Zhang, Chenghao Gu, Heng Li, and Ping Tan. MaskWAM: Unifying mask prompting and prediction for world-action models. *arXiv preprint arXiv:2606.13515*, 2026. 3

[48] Chengbo Yuan, Chuan Wen, Tong Zhang, and Yang Gao. General flow as foundation affordance for scalable robot learning. In *Conference on Robot Learning*, pages 1541–1566, 2025. 3

[49] Shanglin Yuan, Weiheng Zhao, Xin Shi, Haoyi Jiang, Xianda Guo, Liu Liu, Wenyu Liu, Wei Sui, and Xinggang Wang. DreamWAM: Beyond RGB future prediction for world action models. *arXiv preprint arXiv:2608.04996*, 2026. 3

[50] Tianyuan Yuan, Zibin Dong, Yicheng Liu, and Hang Zhao. Fast-WAM: Do world action models need test-time future imagination? In *Advances in Neural Information Processing Systems*, 2026. Accepted for publication. 2, 3, 5

[51] Peng Yun, Shouwang Huang, Hao Li, Jinxi Li, Jianan Wang, and Bo Yang. PhysMani: Physics-principled 3D world model for dynamic object manipulation. In *European Conference on Computer Vision*, pages 20–39, 2026. 2

[52] Yanjie Ze, Gu Zhang, Kangning Zhang, Chenyuan Hu, Muhan Wang, and Huazhe Xu. 3D diffusion policy: Generalizable visuomotor policy learning via simple 3D representations. In *Proceedings of Robotics: Science and Systems*, 2024. 3

[53] Wenyao Zhang, Hongsi Liu, Zekun Qi, Yunnan Wang, Xin-Qiang Yu, Jiazhao Zhang, Runpei Dong, Jiawei He, He Wang, Zhizheng Zhang, Li Yi, Wenjun Zeng, and Xin Jin. DreamVLA: A vision-language-action model dreamed with comprehensive world knowledge. In *Advances in Neural Information Processing Systems*, pages 24195–24228, 2025. 2, 3, 6

[54] Yifan Zhang, Ruiping Wang, and Xilin Chen. Dynamic behavior cloning with temporal feature prediction: Enhancing robotic arm manipulation in moving object tasks. *IEEE Robotics and Automation Letters*, 10(6):5209–5216, 2025. 2

[55] Ruiteng Zhao, Zhengshen Zhang, Yue Su, Wenshuo Wang, Jiahui Li, Zhiyuan Yang, Francis E. H. Tay, Marcelo H. Ang, and Haiyue Zhu. SG-WAM: Self-guided world modeling in geometry-aware policy space. *arXiv preprint arXiv:2608.01397*, 2026. 3

[56] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Proceedings of Robotics: Science and Systems*, 2023. 6

[57] Ruijie Zheng, Jing Wang, Scott Reed, et al. FLARE: Robot learning with implicit world modeling. In *Proceedings of The 9th Conference on Robot Learning*, pages 3952–3971. PMLR, 2025. 3

[58] Chuning Zhu, Raymond Yu, Siyuan Feng, Benjamin Burchfiel, Paarth Shah, and Abhishek Gupta. Unified world models: Coupling video and action diffusion for pretraining on large robotic datasets. In *Proceedings of Robotics: Science and Systems*, 2025. 2

[59] Yuke Zhu, Josiah Wong, Ajay Mandlekar, Roberto Martín-Martín, Abhishek Joshi, Kevin Lin, Abhiram Maddukuri, Soroush Nasiriany, and Yifeng Zhu. robosuite: A modular simulation framework and benchmark for robot learning. *arXiv preprint arXiv:2009.12293*, 2020. 5

[60] Brianna Zitkovich, Tianhe Yu, Sichun Xu, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, pages 2165–2183, 2023. 1